

Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology

Department of Computer Science & Engineering



LAB REPORT

Course Code : CSE 4110
Course Name : Information System Design & Analysis Sessional
Experiment No. : 4
Experiment Name : Structured Analysis of the Existing System

Submitted by		Submitted to
Name	Roll	Emrana Kabir Hashi
Md. Mushfiqur Rahman	2103061	Assistant Professor
Md. Rraf Hasan	2103062	Department of
Md. Raiyan Haque Ashwas	2103063	Computer Science & Engineering,
Md. Asad-Al-Adil	2103064	Rajshahi University of Engineering &
Md. Morakib Hossain	2103065	Technology, Rajshahi.

Date of Submission

13 Jun 2026

Contents

4	Structured Analysis of the Existing System	2
4.1	Introduction	2
4.1.1	Objective of the Experiment	2
4.1.2	Scope of the Application	2
4.2	System Modeling: Data Flow Diagrams	2
4.2.1	Level 0: Context Diagram	2
4.2.2	Level 1: Detailed Process Mapping	3
4.3	Data Specification	4
4.3.1	Data Dictionary	4
4.4	Process Logic Representation	5
4.4.1	Decision Tree Analysis	5
4.4.2	Decision Table Formulation	7
4.5	Database Architecture	7
4.5.1	Entity-Relationship Diagram (ERD)	7
4.5.2	Relational Mapping	8
4.5.3	Entity Definitions and Attributes	10
4.5.4	Relational Mapping	10
4.6	Conclusion	10

4 Structured Analysis of the Existing System

4.1 Introduction

4.1.1 Objective of the Experiment

The primary objective of this experiment is to perform structured systems analysis and database design for a university-wide directory application. This involves translating system requirements and user interactions into standardized Data Flow Diagrams (DFDs) to map the movement of information. Furthermore, the experiment aims to formulate the underlying data structures and logical workflows, culminating in an Entity-Relationship Diagram (ERD) that serves as the architectural blueprint for a relational database implementation.

4.1.2 Scope of the Application

The proposed system operates as a centralized contact and profile directory. Its scope encompasses three primary user domains: read-only access for general mobile application users (students, teachers, and staff), data submission capabilities for authenticated web users, and a moderation interface for system administrators (ICT Cell). The application boundaries include fetching hierarchical category data, rendering individual profiles, managing administrative approvals for new submissions, and interfacing with local device storage to save contact information.

4.2 System Modeling: Data Flow Diagrams

4.2.1 Level 0: Context Diagram

The Context Diagram, also known as a Level 0 Data Flow Diagram, serves as the highest-level abstraction of the application. It defines the absolute system boundary, treating the entire RUET Directory application as a single, encapsulated "black box" process (Process 0.0). The primary objective at this level is to identify all external entities that interact with the system and to map the net flow of data entering and leaving the system boundary.

In this architecture, four primary external entities are identified:

- **App User:** Represents the general campus population (students, teachers, staff) accessing the system in a read-only capacity to browse categories and retrieve specific contact profiles.
- **Web User:** Represents authenticated personnel who submit new profile data or update existing records via the web portal.
- **Admin:** The ICT Cell personnel responsible for governing data integrity by reviewing, approving, or rejecting incoming profile submissions.
- **Device (Phone Contacts):** A hardware-level entity representing the user's local mobile device, receiving data payloads from the application to save contacts directly to the phone's native address book.

By isolating the system at this level, we establish a clear understanding of the application's scope and its external dependencies without being encumbered by internal routing logic.

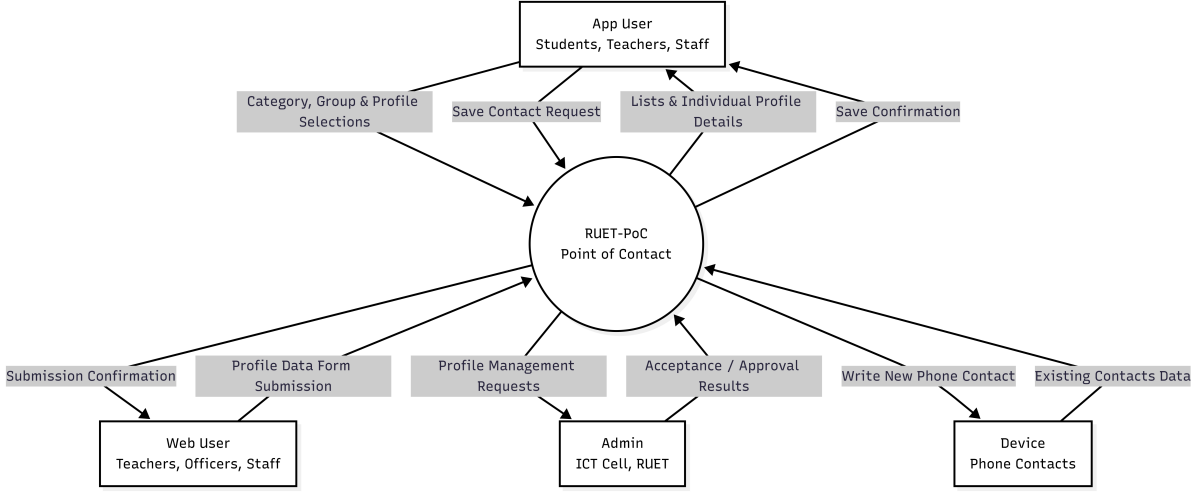


Figure 1: Level 0 Context Diagram of the Directory System

4.2.2 Level 1: Detailed Process Mapping

While the Context Diagram abstracts the system, the Level 1 Data Flow Diagram decomposes the main process into its constituent functional modules. This diagram exposes the internal mechanics of the directory system, mapping exactly how input data is transformed, validated, and stored.

The system is partitioned into six core processes (P1 through P6) that interact with five isolated data stores (DS1 through DS5). This separation of concerns ensures a modular architecture. For instance, the read-only data retrieval pipeline (P1, P2, P3) operates completely independently of the data ingestion pipeline (P4, P5).

When a Web User submits a profile (P4), the data is not immediately published to the user-facing application. Instead, it is routed to a temporary holding store (*DS4: Submitted Profiles*). The administrative workflow (P5) then acts as a gatekeeper. It retrieves pending data from DS4 and, upon validation, migrates the approved records into the permanent public directory (*DS3: Individual Profiles*). This Level 1 mapping is critical for database engineers, as it explicitly defines which processes require read, write, or update permissions for each specific data store.

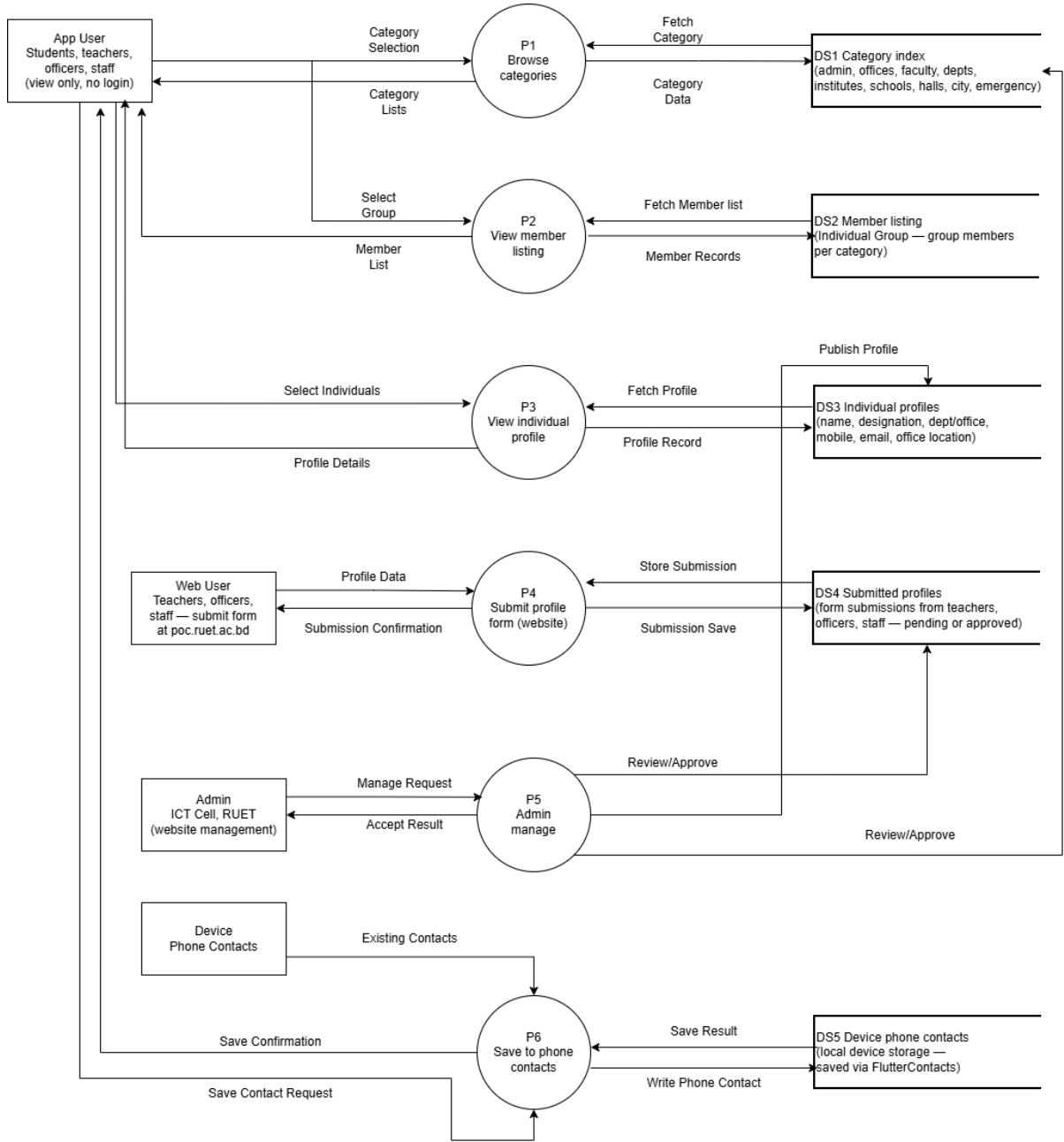


Figure 2: Level 1 Data Flow Diagram showing internal processes and data stores

4.3 Data Specification

4.3.1 Data Dictionary

To ensure data consistency and integrity across the system's architecture, a Data Dictionary is established. This dictionary acts as a centralized repository defining the exact schema, data types, and functional purposes of all elements moving through the system's data flows and residing in the internal data stores.

Data Element	Type	Purpose
Category_Name	VARCHAR	Identifies the category type (e.g., admin, faculty).
Group_ID	INT	Unique identifier for a specific group.
Profile_Name	VARCHAR	Full name of the individual.
Designation	VARCHAR	Job title or role of the individual.
Dept_Office	VARCHAR	Department or office the individual belongs to.
Mobile_Number	VARCHAR	Contact phone number.
Submission_Status	ENUM	Tracks if a profile is 'pending' or 'approved'.

Table 1: Data Dictionary for Core System Elements

4.4 Process Logic Representation

4.4.1 Decision Tree Analysis

In systems analysis, a Decision Tree is a graphical modeling tool used to represent complex, conditional logic in a highly readable, hierarchical format. For this application, the most structurally complex logic occurs within the profile submission and administrative moderation pipeline.

The decision tree maps this workflow as a series of boolean (Yes/No) nodes, starting from the root event (a Web User submitting a form) and terminating at various leaf nodes representing the final state of the data in the backend. Each branch represents a specific logical condition that the system must evaluate sequentially.

By tracing the paths from the root to the leaves, developers can ensure that no "dead ends" or unhandled exceptions exist in the software's logic. For example, if an Admin rejects a profile, the tree dictates a specific operational path—updating the status in DS4 to 'Rejected' without ever triggering the publish command to DS3. This graphical logic representation bridges the gap between abstract business rules and concrete backend programming, ensuring the moderation queue operates deterministically.

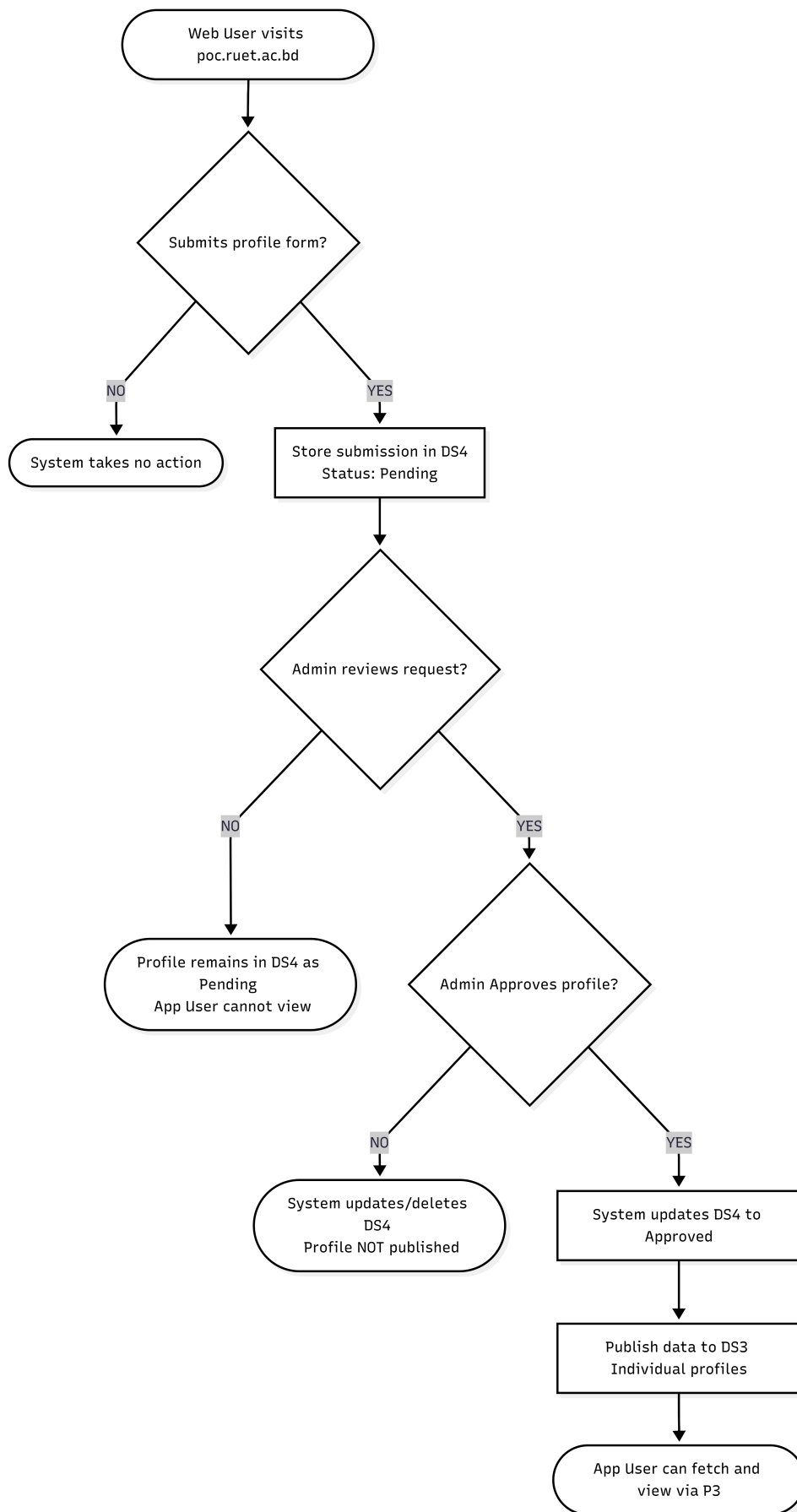


Figure 3: Decision Tree for Profile Submission and Approval

4.4.2 Decision Table Formulation

While the Decision Tree visualizes the flow, the Decision Table provides a rigorous, tabular matrix of all possible logical conditions and their corresponding system actions. This table eliminates ambiguity in the moderation workflow, explicitly defining the exact database operations (such as updating a status flag to 'Approved' or 'Rejected' in DS4) that must execute for every permutation of user and admin inputs.

Conditions / Rules	1	2	3	4
Valid Profile Form Submitted	Y	Y	Y	N
Admin Reviews Request	N	Y	Y	-
Admin Approves Request	-	N	Y	-
Actions				
Store in DS4 (Pending)	X			
Mark as Rejected		X		
Update DS4 (Approved)			X	
Publish to DS3			X	
No Action Taken				X

Table 2: Decision Table for Admin Approval Workflow

4.5 Database Architecture

4.5.1 Entity-Relationship Diagram (ERD)

Transitioning from process modeling to data modeling, the Entity-Relationship Diagram defines the structural schema of the backend database. It translates the isolated data stores identified in the DFD into normalized, interconnected database tables. The ERD illustrates the primary keys, foreign keys, and cardinalities necessary to support the hierarchical nature of the university directory.

- **CATEGORY:** Stores the high-level classifications (e.g., Administration, Faculty, Halls).
 - CategoryID (Primary Key)
 - CategoryName
- **MEMBER_GROUP:** Stores the specific departments or sub-categories within a main category.
 - GroupID (Primary Key)
 - GroupName

- CategoryID (Foreign Key)
- **PROFILE:** The permanent, approved database of individuals visible to the App User.
 - ProfileID (Primary Key)
 - FullName
 - Designation
 - DeptOffice
 - Mobile
 - Email
 - OfficeLocation
 - GroupID (Foreign Key)
- **SUBMISSION:** The temporary holding table for profiles submitted via the website before admin approval.
 - SubmissionID (Primary Key)
 - FullName
 - Designation
 - DeptOffice
 - Mobile
 - Email
 - OfficeLocation
 - Status (Pending / Approved / Rejected)
- **ADMIN:** Stores the credentials for the ICT Cell staff managing the approvals.
 - AdminID (Primary Key)
 - Username

4.5.2 Relational Mapping

The core relationships between the defined entities are as follows:

- **One-to-Many (1:N):** One CATEGORY contains many MEMBER_GROUPS.
- **One-to-Many (1:N):** One MEMBER_GROUP contains many PROFILES.
- **One-to-Many (1:N):** One ADMIN reviews many SUBMISSIONS.
- **One-to-One (1:1):** Once approved, one SUBMISSION becomes one active PROFILE.

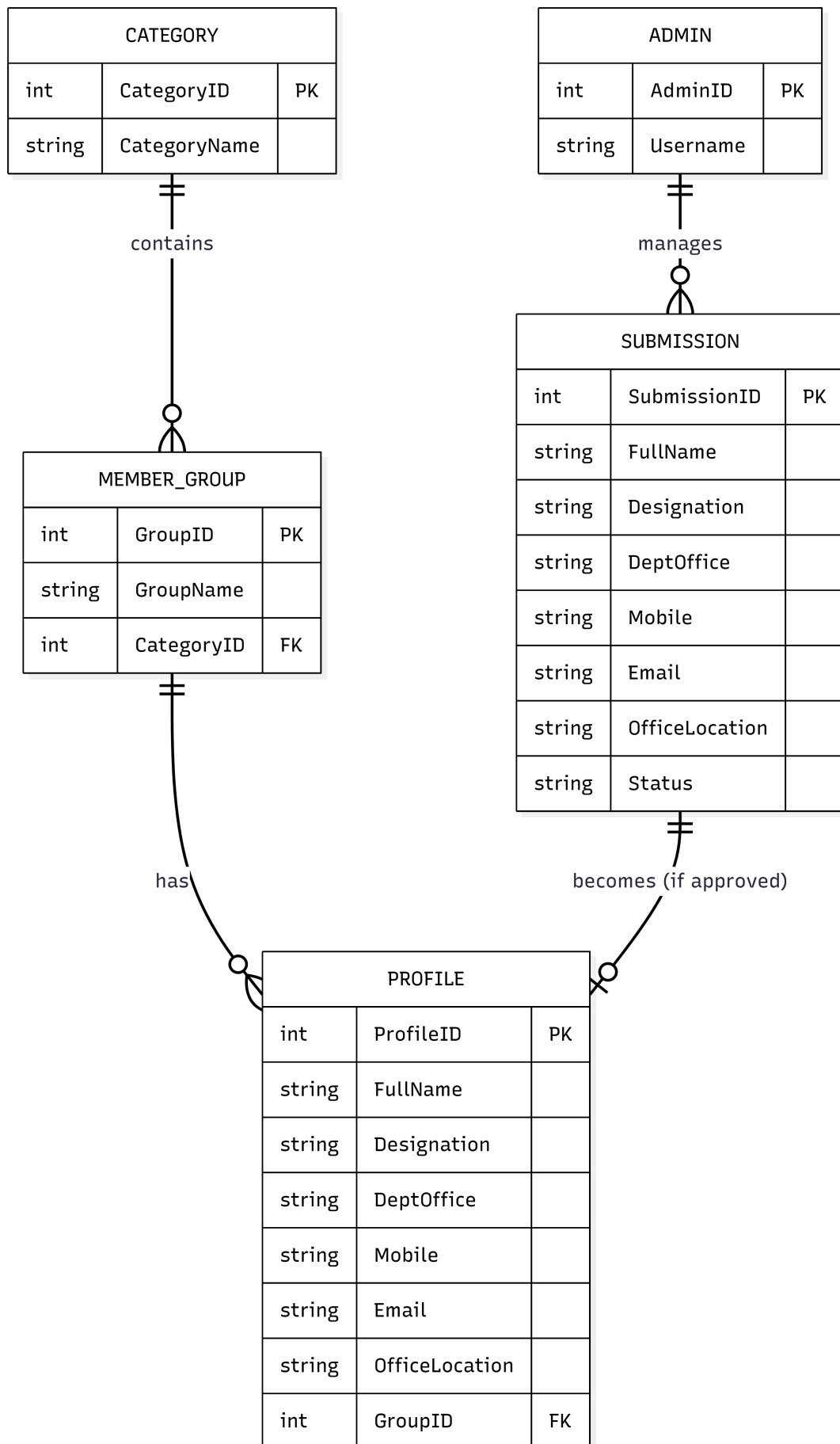


Figure 4: Entity-Relationship Diagram for the Directory Database

4.5.3 Entity Definitions and Attributes

The database architecture is built upon four core entities: `CATEGORY`, `MEMBER_GROUP`, `PROFILE`, and `SUBMISSION`. The `CATEGORY` and `MEMBER_GROUP` entities manage the organizational hierarchy. The `PROFILE` entity acts as the master record for approved individuals, storing attributes such as Full Name, Designation, and Contact Details. The `SUBMISSION` entity mirrors the profile schema but includes a state-tracking attribute to manage the pending approval queue.

4.5.4 Relational Mapping

The relationships between entities are structured to enforce referential integrity. A one-to-many (1:N) relationship exists between `CATEGORY` and `MEMBER_GROUP`, and subsequently between `MEMBER_GROUP` and `PROFILE`, allowing multiple individuals to be assigned to a specific department. A one-to-one (1:1) transformation governs the relationship between the `SUBMISSION` and `PROFILE` tables, where an approved submission record strictly generates a single active profile record.

4.6 Conclusion

This structured analysis successfully deconstructs the directory application from a high-level conceptual model down to a granular relational database schema. By utilizing Data Flow Diagrams, we mapped the necessary routing of user queries and administrative moderation. The subsequent development of decision logic and an Entity-Relationship Diagram provides a mathematically sound, normalized blueprint. This documentation ensures that the transition into the physical coding phase—whether writing SQL schema or implementing backend routing logic—will be built upon a highly organized and scalable architecture.